

基于 LSTM、Transformer、Informer、Autoformer 与 PatchTST 的长时序预测实验报告

执行摘要

长时序预测研究的主线，已经从“能否捕捉远距离依赖”转向“如何在不牺牲计算效率的前提下稳定利用长历史窗口”。在这条演进链上，Informer 通过 ProbSparse 注意力与一次性生成式解码器，把标准 Transformer 的二次复杂度与自回归误差累积问题显式拉到了台面上；Autoformer 则把时间序列分解从预处理提升为网络内部模块，并用 Auto-Correlation 代替点级自注意力；PatchTST 进一步把输入从“单点 token”改成“子序列 patch token”，再配合通道独立建模，在官方监督学习基准上取得了非常强的综合性能。¹

如果只看目前公开、可复核且来源权威的多变量标准 benchmark，PatchTST/42 在 ETT 四个子集、Electricity 或 ECL、Traffic 和 ILI 的主表中都优于 Autoformer 与 Informer；而在 Autoformer 主文公布的子集数据上，普通 LSTM 与这些高效 Transformer 变体之间已经拉开明显差距，尤其在 ETTm2、Traffic 与 ILI 上差距更大。换句话说，现阶段“更强的长序列点预测基线”更接近于 PatchTST，而“最有说服力的多尺度分解证据”则主要来自 Autoformer。²

本报告最重要的实验性结论有三点。第一，多尺度分解不是锦上添花，而是决定长期稳健性的核心机制之一：Autoformer 的消融表明，把渐进式分解嵌入到普通 Transformer 与 Informer 中，到了长步长时能显著降低 MSE，而且简单的“先分解再分别预测”并不稳定，甚至会在超长 horizon 变差。第二，复杂度改进不等于真实吞吐改进：在 Electricity 上，LTSF-Linear 的实测表显示，标准 Transformer 在 96→720 的设定下，推理时间反而比 Informer 和 Autoformer 更短，说明 asymptotic complexity 与 wall-clock efficiency 之间并非线性对应。第三，针对你要求的“外生变量融合与概率预测”，当前这些经典 benchmark 的公开论文并没有给出同一套严格 apples-to-apples 的完整结果矩阵，因此最严谨的做法是把它们视为统一复现实验中的扩展模块，而不是直接拿跨论文数字拼表。³

还需要明确一件事：你指定的预测步长为 24、48、96、168、336，但主流公开长时序论文对 ETT、ECL、Traffic 通常用的是 96、192、336、720，对 ILI 通常用的是 24、36、48、60。也就是说，公开文献里的“官方结果”与“你要求的统一 horizon”并不完全重叠。因此，本报告把内容严格区分成两部分：一部分是公开 benchmark 的复核结果，另一部分是面向 24/48/96/168/336 的统一复现实验方案与可执行建议。⁴

研究边界与关键假设

这份报告的证据来源，优先使用原始论文、官方实现仓库与数据集官网，其次才使用高质量的整理型官方文档。由于当前公开资料里不存在“一篇论文同时覆盖 LSTM、标准 Transformer、Informer、Autoformer、PatchTST，并在 ECL、ETT h1/h2/m1/m2、ILI、Traffic 上统一跑 24/48/96/168/336 全部步长”的完整主表，所以本报告不会伪造一个看似完整、实则不可追溯的总排行榜，而是把“已公开可核验结果”和“推荐统一复现协议”分开。⁵

公开论文之间的数据切分并不完全一致。Informer 对 ETT 使用的是按月份的 12/4/4 划分，对 ECL 使用的是 15/3/4 个月划分；Autoformer 采用 ETT 为 6:2:2、其他数据集为 7:1:2 的按时间顺序切分；Nixtla 的 Long-Horizon 包装文档又把多数数据集描述为 70%/10%/20%，而 ETT 的验证比例另外处理。这意味着即使模型名相同，只要 split 不同、look-back 不同，最终误差就可能出现可观波动。为避免数据泄露，本文一律坚持 chronological split，不做随机打乱。⁶

针对你要求的统一复现实验，本报告采用以下假设，并明确说明对结果的影响。其一，**双轨评估**：文献复核沿用作者原始 horizon 与 split；统一复现则用你要求的 horizon 24/48/96/168/336。其二，**单变量与多变量定义**：多变量采用 $M \rightarrow M$ ，即输入全部通道、输出全部通道；单变量采用 $S \rightarrow S$ ，即只用目标通道作输入并预测目标通道。其三，**滑动长度**：为了兼顾作者原始设定和你指定的 horizon，本报告建议把非 ILI 主任务的 look-back 设成 96、192、336、720 四档，并按 horizon 做映射；ILI 因为官方主设定只到 60 周，因此 96/168/336 周只作为扩展实验，不与文献主表直接横向比较。⁷

若硬件未指定，本报告把“可复现最低建议配置”定为单卡 24GB 显存级别，因为 Autoformer 的公开实验使用单张 TITAN RTX 24GB，Informer 使用单张 V100 32GB。不同 GPU 会显著影响训练吞吐、mixed precision 的收益与显存峰值，但不会改变“哪类结构更能提升长 horizon 稳健性”的总体方法学判断。⁸

数据集与预处理

下表整理了本报告涉及的数据集、原始来源、长时序 benchmark 版本与建议下载入口。需要特别注意的是，长期预测社区里的“Electricity”基本就对应你所写的 ECL；而 Traffic、ILI 在官方公开源上往往不是直接给出论文同款 CSV，需要经过社区通用的 benchmark 整形后才能直接复现主流论文。⁹

数据集	原始官方来源	常用 benchmark 版本	频率	维度	典型时间范围	公开标准 horizon
ETTh1 / ETTh2	ETDataset 官方仓库	ETT-small-h1/h2.csv	1 小时	7 变量	2016/07–2018/07	96/192/336/720
ETTm1 / ETTm2	ETDataset 官方仓库	ETT-small-m1/m2.csv	15 分钟	7 变量	2016/07–2018/07	96/192/336/720
ECL / Electricity	UCI ElectricityLoadDiagrams20112014	常用为 321 条小时级聚合序列	原始 15 分钟，benchmark 常聚合到小时	321 变量	原始 2011–2014；benchmark 常取 2012–2014	96/192/336/720
Traffic	Caltrans PeMS	862 条小时级占用率序列	小时	862 变量	2015–2016	96/192/336/720
ILI	CDC FluView / ILINet	7 变量周级 ILI benchmark	周	7 变量	2002–2021	24/36/48/60

关于数据本体，还需要补充几条对建模最关键的信息。ETDataset 官方仓库说明，ETT-small 含两年数据，既提供 15 分钟版本，也提供小时版本，每个样本点含日期、油温目标 OT 及 6 个负载特征；仓库 README 还明确展示了 ETT 的典型季节性，并指出负载变量呈现明显的日周期与周周期。UCI 的 ElectricityLoadDiagrams20112014 原始数据则是 370 个客户、15 分钟粒度、以 kW 计量，不存在缺失值；长期预测 benchmark 常使用其聚合后的 321 条小时序列版本。Traffic 的长期预测 benchmark 通常是 862 条旧金山湾区高速公路传感器占用率小时序列；ILI benchmark 则来自 CDC FluView 的每周流感样病例比例。¹⁰

对预处理而言，最重要的不是“怎么缩放”，而是**缩放统计量必须只在训练集上估计**。Informer 明确写到了其输入先做 zero-mean normalization；PatchTST 则在模型内部使用 instance normalization 来缓和 train-test

distribution shift；而 LTSF-Linear 进一步指出，长时序 benchmark 中存在明显的分布漂移问题，尤其在 ETTh1、ETTh2、ILI 这类数据里更值得注意。基于这些证据，统一复现实验建议采用三层预处理：先按时间顺序切分，再在训练集统计 z-score 或 robust scaler，最后在模型侧对 PatchTST 类结构额外启用 instance normalization。¹¹

在滑窗构造上，建议把一个样本写成

$$X_t \in \mathbb{R}^{L \times C}, \quad Y_t \in \mathbb{R}^{H \times C}$$

其中 L 为 look-back, H 为 horizon, C 为通道数。对多变量任务，用全部通道构造 X_t 并预测全部通道；对单变量任务，只保留目标通道。若要做外生变量融合，可以把时间特征、节假日、工作日、气象字段或流行病学周编号编码为 Z_t ，再按 concat、cross-attention 或 conditional encoding 的方式并入主干网络。TFT 的摘要明确指出，多步预测往往同时包含静态变量、未来已知变量与仅历史可见的外生时间序列，因此“只把所有变量硬拼在一起”并不是唯一合理方案。¹²

对你要求的 24/48/96/168/336 统一步长，建议采用下面这套**统一滑窗假设**。对 ETT、ECL、Traffic：

24 → L=96，48 → L=192，96 → L=336，168 → L=720，336 → L=720。

这样既保留了“历史窗口至少覆盖 4 个 horizon”的经验，也尽可能靠近文献中最常见的 `seq_len ∈ {96, 192, 336, 720}`。对 ILI：保留文献标准的 24/36/48/60 作为主表，扩展实验再尝试 96/168/336，但必须单独报告，因为它已经脱离了公开 benchmark 共识。这个约束不是形式主义，而是因为 ILI 的标准 horizon 本来就显著短于高频能源和交通数据。⁷

方法与模型实现细节

本报告比较的五个主干模型，实际上代表了三种完全不同的长依赖建模思想。LSTM 依赖门控递归状态在时间上逐步传播记忆；标准 Transformer 依赖全局自注意力直接连接任意两个时间位置；Informer、Autoformer 与 PatchTST 则分别从“稀疏注意力”“可学习分解 + 周期相关性”“patch token + 通道独立”三个方向改写了标准 Transformer 的时序 inductive bias。Transformer 原论文强调其完全基于注意力机制来建模输入输出全局依赖；LSTM 原始论文强调通过常数误差环路与门机制缓解长时间跨度上的梯度衰减；Informer、Autoformer、PatchTST 则都是围绕长时序预测场景对这两种范式的实用化改造。¹³

flowchart LR

A[原始时间序列 CSV] --> B[按时间顺序切分]

B --> C[滑窗采样]

C --> D{可选增强模块}

D --> D1[多尺度分解]

D --> D2[外生变量编码]

D --> D3[概率预测头]

D1 --> E[主干模型]

D2 --> E

E --> F[点预测或分位数预测]

D3 --> F

F --> G[MSE / MAE / MAPE]

F --> H[区间宽度 / 覆盖率]

G --> I[误差衰减与复杂度分析]

H --> I

从实现角度看，五个模型可以统一到一个训练接口中。LSTM 建议用 2 层 encoder-only 或 encoder-decoder 结构，隐藏维度 128 到 256，最后接一个 direct multi-output 线性头，避免 recursive rollout；标准

Transformer 则建议直接采用 LTSF-Linear 文中所说的“与 Autoformer 代码基线一致的 DMS 版 vanilla Transformer”，因为这样才能把它和 Informer、Autoformer 放到尽量一致的训练脚本里比较。LTSF-Linear 的附录明确写到，其 vanilla Transformer、Informer 和 Autoformer 实现都来自 Autoformer 代码体系；这对于做统一自定义实验尤其重要。¹⁴

Informer 的关键结构是 ProbSparse attention、自注意力 distilling 和一次性生成式 decoder。原论文指出，vanilla Transformer 的自回归解码在长步长下会带来速度瓶颈与误差传播，而 Informer 通过 generative-style decoder 在一次前向传播中直接输出整个预测跨度，同时用 ProbSparse 把注意力复杂度降到近似 $O(L \log L)$ 。这使它成为“高效长序列 Transformer”路线的起点。¹⁵

Autoformer 的实现重点不是“把普通注意力换成另一个稀疏注意力”这么简单，而是同时引入了渐进式分解 block 与 Auto-Correlation 机制。主文给出的实现细节包括：输入长度对 ILI 取 36、对其他数据集取 96；模型含 2 层 encoder 和 1 层 decoder；训练使用 Adam，batch size 为 32，早停阈值是 10 个 epoch，单卡 TITAN RTX 24GB，重复 3 次。作者还在附录中指出，ETT 这类周期明显的数据常常只需要 96 的 look-back，而 ILI 这种“没有明显周期”的数据则更依赖更长历史。¹⁶

PatchTST 的实现方式与前两者最不一样。它不是从注意力稀疏性入手，而是先把一条时间序列切成 patch，再把 patch 视为 Transformer token，因此 token 数从 L 变成近似 N_p ，注意力计算与显存开销按 token 数二次缩减。与此同时，PatchTST 使用 channel-independence：每个通道各自进同一个 Transformer 骨干并共享参数。论文正文明确说明，这样做带来三个收益：保留局部语义、显著减少注意力图的计算与内存、允许模型看到更长 look-back；而官方仓库则提供了 PatchTST/42 作为默认监督学习模型，并把 patch length、stride、look-back、prediction length 都暴露为可调超参数。¹⁷

在你关心的三类增强模块里，多尺度分解最有现成证据。经典 STL 把序列分解为 trend、seasonal 和 remainder；EMD 把非线性、非平稳信号自适应分解为若干 IMF；SSA 则利用轨迹矩阵与谱分解来提取趋势、振荡和噪声成分。Autoformer 的重要贡献不在于“使用分解”本身，而在于把分解嵌入到了深网络内部，变成可端到端学习的 progressive decomposition。对于统一实验，我建议把分解 ablation 做成四档：无分解、外部 STL、外部 EMD 或 SSA、内部 learnable decomposition。¹⁸

外生变量融合方面，当前这些经典 benchmark 的公开主表并没有专门强调 heterogeneous covariates，因此最好的方法论参考反而是 TFT。TFT 的摘要明确指出，多步预测通常混合了静态协变量、未来已知输入和只在历史中出现的外生序列；它用 recurrent 层处理局部模式、用 self-attention 处理长依赖，并加入特征选择与 gating 机制来抑制无关输入。基于这条思路，本报告建议比较三种融合方式：直接拼接 concat、双编码器 cross-attention、基于外生上下文的 conditional encoding 或 gating。若外生信息只是简单时间戳特征，concat 通常已经足够；若外生信息异质且未来已知，cross-attention 或条件编码更有意义。¹²

概率预测方面，建议同时实现两条路线。第一条是 **MC Dropout**：训练时照常做 point forecast，测试时保持 dropout 开启，重复采样 N 次，由样本均值给出点预测、由经验分位数给出置信区间。Gal 与 Ghahramani 的工作把这种做法解释为一种近似贝叶斯推断。第二条是 **深度分位回归**：输出多个分位点，例如 $q \in \{0.1, 0.5, 0.9\}$ ，用 pinball loss 优化；TFT 和 DeepAR 都强调了概率预测在多步预测场景中的重要性。就工程可操作性而言，MC Dropout 改动最小，但推理成本会乘以采样次数；分位回归训练成本更稳定，且更适合直接输出区间。¹⁹

实验设计

为了同时满足“学术可比性”和“你指定的统一协议”，最稳妥的设计是分成两条实验线。**文献复核线**严格沿用原论文的公开 benchmark，用来回答“这些模型在社区共识设置下各自表现如何”；**统一复现线**则严格用 24/48/96/168/336、chronological split、统一滑窗与统一外生变量接口，用来回答“当实验约束被统一之后，结论是否还成立”。前者保证结论不脱离现有文献，后者保证你的报告可执行。²⁰

统一复现线建议采用如下训练协议。损失函数方面，点预测主任务使用 MSE，辅报 MAE 和 MAPE；概率预测额外使用 pinball loss，并报告区间覆盖率与区间宽度。优化器统一用 AdamW；学习率可在 $[10^{-4}, 3 \times 10^{-4}, 10^{-3}]$ 搜索，调度器建议 cosine decay 或 one-cycle；早停 patience 建议 10 到 20 个 epoch；随机种子至少 3 个。这里把 Autoformer 公开的 batch size=32、早停 10 epoch 当作一个安全起点，而对 PatchTST 可在显存允许时把 batch size 往 64 调。 ²¹

对于 look-back、模型容量与 patch 相关超参，本报告给出一套可直接执行的搜索表。这里的“最终建议值”是工程建议，不是声称文献唯一最优值。

超参数	搜索范围	建议默认值	说明
seq_len	{96, 192, 336, 720}	336	非 ILI 主设定，兼顾稳定性与显存
seq_len_ili	{24, 36, 48, 60}	36	与 Autoformer 主设定对齐
d_model	{128, 256, 512}	256	小中型 benchmark 足够稳妥
n_heads	{4, 8}	8	Transformer 类默认
e_layers	{2, 3, 4}	2 或 3	Autoformer 公开为 2 encoder；其他可取 3
dropout	{0, 0.1, 0.2, 0.3}	0.1	同时服务于 MC Dropout
patch_len	{8, 16, 24, 32}	16	PatchTST 建议从中等 patch 起步
stride	{4, 8, 12, 16}	8	兼顾 token 压缩与局部细节
decomp_kernel	{13, 25, 49}	25	与 DLinear/Autoformer 常见设置接近
quantiles	{(0.1,0.5,0.9), (0.05,0.5,0.95)}	(0.1,0.5,0.9)	先追求稳定，再追求窄区间
mc_samples	{20, 30, 50}	30	训练开销不变，推理开销可控

消融实验建议按模块化矩阵组织，而不是每个模型各写一套脚本。最简洁的设计是：

- A** 主干模型 only；
- B** 主干 + 多尺度分解；
- C** 主干 + 外生变量 concat；
- D** 主干 + 外生变量 cross-attention；
- E** 主干 + conditional encoding；
- F** 主干 + quantile head；
- G** 主干 + MC Dropout；
- H** 主干 + 最优分解 + 最优外生融合 + 最优概率头。

这样可以同时回答三类问题：哪个增强模块最稳定、不同主干对增强模块的敏感性是否一致、哪些组合会产生冲突。这个矩阵对 PatchTST、Autoformer 尤其有意义，因为前者缺少显式分解，后者缺少强 patch token 化。 ²²

为保证复现，训练脚本可以统一成下面这个伪代码框架。它既支持 point forecast，也支持 quantile head 和 MC Dropout。

```

for model_name in ["lstm", "transformer", "informer", "autoformer", "patchtst"]:
    for dataset in datasets:
        train_ds, val_ds, test_ds = make_chrono_split(dataset, split_cfg)

```

```

scaler = fit_scaler(train_ds)
train_loader = make_windows(train_ds, seq_len=L, pred_len=H, scaler=scaler)
val_loader   = make_windows(val_ds,   seq_len=L, pred_len=H, scaler=scaler)
test_loader  = make_windows(test_ds,  seq_len=L, pred_len=H, scaler=scaler)

model = build_model(
    backbone=model_name,
    decomposition=decomp_cfg,
    exogenous_fusion=exo_cfg,
    probabilistic_head=prob_cfg,
)

best = train_with_early_stopping(
    model=model,
    train_loader=train_loader,
    val_loader=val_loader,
    optimizer="adamw",
    scheduler="cosine",
    loss=select_loss(prob_cfg),
    patience=patience,
    seed=seed,
)

test_metrics = evaluate_point_metrics(best, test_loader)  # MSE / MAE / MAPE
if prob_cfg == "quantile":
    test_metrics |= evaluate_quantile_metrics(best, test_loader)
if prob_cfg == "mc_dropout":
    test_metrics |= evaluate_mc_dropout(best, test_loader, n_samples=30)

save_predictions(best, test_loader)
profile_complexity(best, sample_batch)

```

图表部分，建议最少固定输出五类：预测值与真实值曲线、残差时序图、误差随 horizon 变化折线、不同模型的箱线图、复杂度-精度 Pareto 图。下面的代码片段可以直接做前三类图，输入是一个包含 `y_true`、`y_pred`、`timestamp` 的 DataFrame，以及一个保存各 horizon 指标的 DataFrame。

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

def plot_forecast(df, title="forecast vs truth"):
    plt.figure(figsize=(10, 4))
    plt.plot(df["timestamp"], df["y_true"], label="truth")
    plt.plot(df["timestamp"], df["y_pred"], label="pred")
    plt.title(title)
    plt.legend()
    plt.tight_layout()
    plt.show()

def plot_residual(df, title="residual over time"):

```

```

resid = df["y_true"] - df["y_pred"]
plt.figure(figsize=(10, 4))
plt.plot(df["timestamp"], resid)
plt.axhline(0.0, linestyle="--")
plt.title(title)
plt.tight_layout()
plt.show()

def plot_error_horizon(metrics_df, metric="MSE", title="error vs horizon"):
    plt.figure(figsize=(6, 4))
    for model_name, g in metrics_df.groupby("model"):
        g = g.sort_values("horizon")
        plt.plot(g["horizon"], g[metric], marker="o", label=model_name)
    plt.xlabel("horizon")
    plt.ylabel(metric)
    plt.title(title)
    plt.legend()
    plt.tight_layout()
    plt.show()

```

复杂度画像也应流程化，而不是最后手工补表。若使用 PyTorch，可以把参数量、MACs、推理时间与显存峰值统一写成一个 profiler 函数。LTSF-Linear 对 Electricity 的公开表已经表明，参数量和 MACs 并不能完全预测 wall-clock time；因此实际 profiling 不能省。 ²³

结果与可视化

下面先给出公开主表中最可比的一部分。如果关注多变量 point forecast，PatchTST 论文的监督学习表最有价值，因为它在同一张表里给出了 PatchTST/64、PatchTST/42、DLinear、FEDformer、Autoformer、Informer、Pyraformer、LogTrans；横向可比性好于把不同论文的最佳数字机械拼接。需要同时说明：公开主表普遍只提供 MSE/MAE，不提供 MAPE，因此你要求的 MAPE 必须在统一复现实验中另外补算。 ²⁴

数据集	Horizon	PatchTST/42	Autoformer	Informer
ETTh1	96	0.375 / 0.399	0.435 / 0.446	0.941 / 0.769
ETTh1	336	0.431 / 0.436	0.486 / 0.487	1.038 / 0.784
ETTh2	96	0.274 / 0.336	0.332 / 0.368	1.549 / 0.952
ETTh2	336	0.331 / 0.380	0.477 / 0.479	4.215 / 1.642
ETTM1	96	0.290 / 0.342	0.510 / 0.492	0.626 / 0.560
ETTM1	336	0.366 / 0.392	0.510 / 0.492	1.005 / 0.741
ETTM2	96	0.165 / 0.255	0.205 / 0.293	0.355 / 0.462
ETTM2	336	0.278 / 0.329	0.343 / 0.379	1.270 / 0.871
ECL	96	0.130 / 0.222	0.196 / 0.313	0.304 / 0.393
ECL	336	0.167 / 0.261	0.214 / 0.327	0.333 / 0.422
Traffic	96	0.367 / 0.251	0.597 / 0.371	0.733 / 0.410

数据集	Horizon	PatchTST/42	Autoformer	Informer
Traffic	336	0.398 / 0.265	0.623 / 0.387	0.776 / 0.434
ILI	24	1.522 / 0.814	2.906 / 1.182	4.657 / 1.449
ILI	48	1.673 / 0.854	3.024 / 1.145	5.004 / 1.542

从这个表里可以读出三个稳定事实。第一，PatchTST/42 在你关心的所有公开 benchmark 主任务上都比 Autoformer 与 Informer 更强，尤其在 Traffic、ECL 和 ILI 上优势非常大。第二，Informer 虽然在“高效 Transformer”研究史上地位很高，但在后续更强基线面前，点预测精度已经明显落后。第三，Autoformer 仍然是非常强的次优基线，特别是在 ETT 与 ECL 这类趋势和周期都明显的数据集上，它与 PatchTST 的差距并不像与 Informer 那样夸张。 ²⁵

普通 LSTM 的公开数据要少得多。Autoformer 主文在 ETTm2、Electricity、Traffic、ILI 上公布了 plain LSTM 的多变量结果；这些结果已经足够说明：一旦 horizon 拉长到 benchmark 主设定，LSTM 的误差普遍明显高于现代 direct multistep Transformer 变体。由于 ETT1、ETT2、ETTm1 上没有在同一来源里给出 plain LSTM 的官方主表，本报告不把所有 recurrent baseline 冒充为 LSTM。 ²⁶

数据集	Horizon	LSTM	Autoformer	Informer
ETTM2	96	2.041 / 1.073	0.255 / 0.339	0.365 / 0.453
ETTM2	336	2.568 / 1.238	0.339 / 0.372	1.363 / 0.887
ECL	96	0.375 / 0.437	0.201 / 0.317	0.274 / 0.368
ECL	336	0.439 / 0.473	0.231 / 0.338	0.300 / 0.394
Traffic	96	0.843 / 0.453	0.613 / 0.388	0.719 / 0.391
Traffic	336	0.853 / 0.455	0.622 / 0.337	0.777 / 0.420
ILI	24	5.914 / 1.734	3.483 / 1.287	5.764 / 1.677
ILI	48	6.736 / 1.857	2.669 / 1.085	4.763 / 1.469

如果把这些数值转成易懂的结论，可以得到更细的判断。在 ECL 的 96 步上，PatchTST/42 的 MSE 为 0.130，而 Autoformer 与 Informer 分别为 0.196 与 0.304；在 Traffic 的 96 步上，PatchTST/42 为 0.367，而 Autoformer 与 Informer 分别为 0.597 与 0.733；在 ILI 的 24 步上，PatchTST/42 为 1.522，而 Autoformer 为 2.906、Informer 为 4.657。也就是说，PatchTST 在强多变量 benchmark 上并不是“微弱领先”，而是很多时候形成了一个误差层级。另一方面，LSTM 在 ETTm2 和 ILI 上的误差量级已经明显落后，这也是为什么现代长时序研究更倾向于把 LSTM 视为资源受限、概率建模或工程回归的参考线，而不是争夺最高精度的主战模型。 ²⁷

关于标准 Transformer，需要比对得更诚实一些：在我核查到的原始论文与官方仓库中，并没有一张完整主表同时公开它在你指定所有数据集上的点预测误差；但 LSTF-Linear 给出了它在 Electricity、96→720 的实际参数量、MACs、推理时间和显存，还明确说明 vanilla Transformer、Informer 与 Autoformer 使用的是同一套 Autoformer 代码系实现。因此，标准 Transformer 在这份报告中的角色更准确地说是“统一复现实验中的必要基线”，而不是“文献中现成可抄的完整精度主表”。 ¹⁴

消融、误差累积与长期预测衰减分析

关于多尺度分解，Autoformer 的消融是本题最有力的直接证据。作者在多变量 ETT 上，把普通 Transformer、Informer、LogTrans、Reformer 依次接上两种分解策略：一种是传统的“先分解再分别预测”，一种是 Autoformer 风格的 progressive decomposition。结果并不是“只要分解都好”，而是“嵌入式渐进分解明显优于外部分解，而且 horizon 越长优势越大”。²⁸

预测长度	Transformer 原始	Transformer 先分解	Transformer 渐进分解	Informer 原始	Informer 先分解	Informer 渐进分解
96	0.604	0.311	0.204	0.365	0.490	0.354
192	1.060	0.760	0.266	0.533	0.658	0.432
336	1.413	0.665	0.375	1.363	1.469	0.481
720	2.672	3.200	0.537	3.379	2.766	0.822

这张表的含义非常强。对普通 Transformer 而言，predict-720 时 MSE 从 2.672 直接降到 0.537；对 Informer 而言，predict-720 从 3.379 降到 0.822。更关键的是，“先分解再各自预测”并不稳定：Transformer 在 720 上甚至从 2.672 恶化到了 3.200，Informer 在 336 上也从 1.363 变成了 1.469。由此可以得到一个对你课题非常重要的实验结论：**多尺度时序分解应优先设计为网络内部、可联训的结构，而不是固定预处理**。²⁸

长 horizon 下的误差衰减，也可以从公开主表中得到一些非常清楚的模式。先看 Traffic: PatchTST/42 的 MSE 从 96 步的 0.367 增长到 720 步的 0.434，Autoformer 从 0.597 增长到 0.639，Informer 从 0.733 增长到 0.827。再看 ECL: PatchTST/42 从 0.130 增长到 0.202，Autoformer 从 0.196 增长到 0.236，Informer 从 0.304 增长到 0.351。这里可以得到一个细粒度判断：**PatchTST 往往具有最低误差底座，而 Autoformer 在部分周期型数据上呈现更平缓的绝对退化率；Informer 两者都不占优**。²⁵

如果回到“误差累积”这个更本质的问题，LTSF-Linear 对 IMS 与 DMS 的区分非常关键。该文明确把 iterated multi-step forecasting 和 direct multi-step forecasting 区分开来，并指出前者天然面临 error accumulation；Informer 之所以重要，正是因为它用 generative-style decoder 规避了一个一个递推输出的成本与累积偏差。对于你的统一实验，我强烈建议把误差累积单列为一个 ablation：对 LSTM 和标准 Transformer 同时实现 recursive one-step 与 direct multi-output 两种 head，再比较 horizon 增长时的误差斜率。这比只比较一个最终 MSE 更能说明“为什么某些结构在长时序上会塌”。²⁹

季节性和周期性讨论也必须数据集特异化。ETDataset 官方 README 明确展示了 ETT 的显著季节趋势，并指出负载变量有明显日周期和周周期；Autoformer 的输入长度分析则进一步指出，ETT 这类周期明显的数据在 96 的 look-back 下已经能得到不错表现，而 ILI 这种“没有明显周期”的数据则更依赖更长历史输入。作者还在超参数敏感性分析里指出，ETT 与 Traffic 这类周期较明显的数据倾向于选择更大的 Auto-Correlation factor，而 ILI 过大的 factor 反而可能引入噪声。对你的课题而言，这意味着“分解 + patching + exogenous calendar covariates”并不是对所有数据一视同仁地加，而是应在周期型数据和弱周期型数据上采用不同优先级。³⁰

在外生变量融合与概率预测方面，公开 benchmark 没有现成的一张同表消融，但我们仍然可以给出足够严谨的扩展实验结论框架。其一，在这些 benchmark 的公开 CSV 中，除时间戳和已有多变量观测外，额外未来可知外生变量并不丰富，因此若你不额外接入节假日、天气或流行病学背景数据，concat、cross-attention、conditional encoding 之间的差别往往会被弱化。其二，若外生变量确实是 heterogenous 且未来已知，例如日历、假日、流感季节标签，那么 TFT 所展示的 gating 和 feature selection 机制很值得移植到 PatchTST 或 Autoformer 的输入端。其三，概率预测建议优先比较 quantile head 与 MC Dropout：前者更适合多步区间输出，后者更适合作为不确定性 sanity check。这里的判断属于“基于原始方法机制的设计推断”，不是声称已有公开主表已给出全部数值。³¹

复杂度、资源消耗与超参数经验

关于复杂度，不同模型解决的是不同瓶颈。标准 Transformer 的核心瓶颈是全量 attention 的二次复杂度；Informer 通过 ProbSparse 与 distilling 追求近似 $O(L \log L)$ ；Autoformer 以 Auto-Correlation 取代点级 self-attention，并在摘要中强调其固有的 $O(L \log L)$ 复杂度；PatchTST 则把复杂度从“对 L 个时间点做二次注意力”改写成“对 N_p 个 patch token 做二次注意力”，其中 $N_p \ll L$ 。PatchTST 正文直接指出，patching 使输入 token 数从 L 近似减少到 L/S ，从而让注意力图的计算和显存按 stride 的平方阶缩减。 ³²

实际资源消耗则更微妙。LTSF-Linear 在 Electricity、 $L = 96$ 、 $T = 720$ 的设置下给出了统一实测：标准 Transformer 为 13.61M 参数、4.03G MACs、26.8ms 推理时间、6091MiB 显存；Informer 为 14.39M、3.93G、49.3ms、3869MiB；Autoformer 为 14.91M、4.41G、164.1ms、7607MiB。这个结果非常值得在报告中强调，因为它说明“理论更高效”不一定对应“实际更快”；Autoformer 在这个设定下甚至比 vanilla Transformer 更慢、更吃显存。另一方面，PatchTST 的论文 case study 又给出了另一个维度的证据：在 Traffic、Electricity 和 Weather 上，patching 相比不打 patch 的设置分别带来了约 22 倍、19 倍和 4 倍的运行时间收益。两类证据放在一起，结论就非常明确：**真正应该优化的是 tokenization + kernel 实现 + decoder 方式的整体系统，而不只是纸面复杂度。** ³³

模型	主要复杂度瓶颈	理论趋势	公开实测或公开证据	工程解读
LSTM	时间递归串行	近似 $O(Ld^2)$	本报告未找到同条件官方统一资源表	低参数、强顺序归纳偏置，但长 horizon 吞吐差
Transformer	全量 self-attention	$O(L^2)$	13.61M, 4.03G MACs, 26.8ms, 6091MiB	作为统一基线必不可少，但长序列会压显存
Informer	稀疏注意力 + distilling	近似 $O(L \log L)$	14.39M, 3.93G, 49.3ms, 3869MiB	显存更省，但 wall-clock 不一定更快
Autoformer	分解 + Auto-Correlation	近似 $O(L \log L)$	14.91M, 4.41G, 164.1ms, 7607MiB	长期稳健，但实现开销和 kernel 代价较高
PatchTST	patch token 数 二次注意力	$O(N_p^2)$, $N_p \ll L$	patching 在个别 case 上带来 4-22 倍时间收益	在点预测上综合最强，且特别适合更长 look-back

对超参数经验，本报告最推荐记住三条。第一，**look-back 不存在绝对最优，要看数据是否有清晰周期。**Autoformer 的附录已经很明确：ETT 类数据 96 的输入窗口往往够用，ILI 则更依赖更长输入。第二，**公式级复杂度优化不等于实际更快**，因此所有结果都应带上统一 profiler，而不是只报参数量。第三，**PatchTST 的 patch 不是越大越好**：patch 太大容易模糊局部模式，patch 太小又失去 token 压缩收益，因此建议把 `patch_len=16, stride=8` 一类的中等配置作为起点，再做小规模网格搜索。最后这一条是工程建议，不是声称文献唯一结论。 ³⁴

如果必须给出“在你这个课题下最实用的调参顺序”，我会建议按下面的顺序进行。先定 split 与 horizon，保证 no leakage；再只调 `seq_len` 和主干 backbone，因为它们决定上限；然后才调 decomposition kernel 与 patch 配置；外生变量融合在有真实未来可知 covariates 时再引入；概率预测最后加，因为它更像输出层设计，而不是主干建模能力。这个顺序能够显著减少无效搜索。它背后的依据是：Autoformer 的输入长度敏感性和 PatchTST 的 tokenization 设计，都明显比单纯换一个学习率更影响最终表现。 ³⁵

结论

综合现有公开证据，可以给出一个相当清晰的判断。若目标是**长时序点预测精度**，在你列出的模型中，PatchTST 是当前最强的公开基线；若目标是**解释为什么长期预测会稳定或衰减**，Autoformer 的“渐进式分解 + Auto-Correlation”提供了最扎实的机制证据；Informer 的价值主要体现在它率先系统性地解决了长时序的稀疏注意力与 direct multistep 解码问题，但在今天的公开 benchmark 上，点预测精度已经普遍落后于 PatchTST 和多数设定下的 Autoformer；LSTM 则更适合作为低资源、递归式或概率建模的参考底线，而不是当前长 horizon 争榜的主力。³⁶

如果把你的三个增强方向也纳入结论，那么最值得推荐的统一方案是：**PatchTST 主干 + 可学习多尺度分解 + 条件式外生变量编码 + 分位数预测头**，并把 **MC Dropout** 作为不确定性校准的交叉检查工具。原因在于，PatchTST 已经证明了 patch tokenization 与 channel-independence 对长期点预测足够强，而 Autoformer 的分解消融又证明“嵌入式分解”对长 horizon 稳健性非常关键；TFT 则为 heterogeneous exogenous covariates 的条件选择与 gating 提供了成熟范式。对于 ETT、Traffic、ECL 这类周期型数据，分解与 patching 的收益通常最大；对于 ILI 这类弱周期、样本更短、噪声更强的数据，合适的 look-back 与外生上下文往往比更激进的结构复杂化更重要。³⁷

最后，必须把本报告中最重要“方法学限制”再强调一次。第一，公开主表与您指定的统一 horizon 并不完全重叠，尤其 ILI 的官方标准只有 24/36/48/60。第二，标准 Transformer 与 plain LSTM 没有在所有目标数据集上都给出统一、官方、同条件的公开主表，因此它们在这份报告里更像“必须重跑的统一基线”。第三，公开 benchmark 主表几乎都只报 MSE/MAE，不报 MAPE，更不系统报概率校准指标。因此，要把这份分析真正落到你的课题要求上，最有价值的下一步不是再搜更多论文，而是按本文给出的统一脚本与双轨评估协议，把 24/48/96/168/336 的复现实验跑出来。届时，结论很可能仍然成立，但会更加严格，也更贴近你的目标问题。³⁸

¹ ¹⁵ <https://arxiv.org/abs/2012.07436>

<https://arxiv.org/abs/2012.07436>

² ¹⁷ ²⁰ ²⁴ ²⁵ ²⁷ ³⁶ ³⁷ <https://arxiv.org/pdf/2211.14730>

<https://arxiv.org/pdf/2211.14730>

³ ⁵ ⁷ ⁸ ¹⁶ ²¹ ²² ²⁶ ²⁸ ³⁴ ³⁵ <https://arxiv.org/pdf/2106.13008>

<https://arxiv.org/pdf/2106.13008>

⁴ ³⁸ https://nixtlaverse.nixtla.cn/datasetsforecast/long_horizon.html

https://nixtlaverse.nixtla.cn/datasetsforecast/long_horizon.html

⁶ ¹¹ <https://arxiv.org/pdf/2012.07436>

<https://arxiv.org/pdf/2012.07436>

⁹ ¹⁰ ³⁰ <https://github.com/zhouhaoyi/ETDataset>

<https://github.com/zhouhaoyi/ETDataset>

¹² ³¹ <https://arxiv.org/abs/1912.09363>

<https://arxiv.org/abs/1912.09363>

¹³ ³² <https://arxiv.org/abs/1706.03762>

<https://arxiv.org/abs/1706.03762>

¹⁴ ²³ ²⁹ ³³ <https://arxiv.org/pdf/2205.13504>

<https://arxiv.org/pdf/2205.13504>

¹⁸ <https://otexts.com/fpp3/stl.html>

<https://otexts.com/fpp3/stl.html>

¹⁹ <https://arxiv.org/abs/1506.02142>

<https://arxiv.org/abs/1506.02142>